

Phase 3 명세 — 리소스 수명 재배포

Lifetime Relocation · 코드는 직접 구현 · 이 문서 = 무엇을 어떻게 바꿀지 + 도식

전제: Phase 2(레지스트리+스코프 골격) + R2/S1/S2/R1/S3(계측 배선·정리) 완료·검증됨. 수치는 Phase 2까지 무관, Phase 3에서 모든 정량 성과가 발생한다.

0. 한 줄 요약

2버킷 시절 "애매하면 안전하게 isGlobal:true" 관성으로 거의 모든 리소스가 영구 상주 중이다. Phase 2에서 깬 스코프 골격 위에서 각 리소스를 제 실제 수명 스코프로 재배포해 상주 메모리를 회수한다. 재배포 = 호출부의 isGlobal:true를 진짜 수명 스코프로 바꾸는 것 — refCount가 자동 정산하므로 수동 Release 없음(HP바 누수를 만든 방식의 구조적 대체).

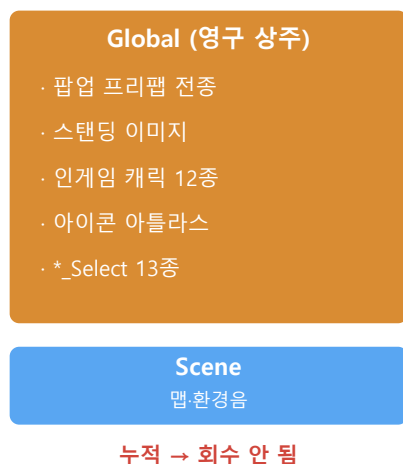
1. 현 상태 진단 (grep 근거)

거의 모든 로드가 isGlobal:true로 Global에 영구 적재된다:

호출부	로드 대상	현재	진짜 수명
UIManager.ShowPopupUIAsync:117	팝업 프리팹 전종	Global	씬
UI_Info.SetCharacterStandingImage:95	스탠딩 이미지(대형)	Global	팝업
UI_EscapeMenu:47	스탠딩 이미지	Global	팝업
GameScene:172	인게임 캐릭 프리팹	Global	파티
GetSpriteFromAtlasAsync:177	아이콘 아틀라스	Global	소비자 수명
StartScene:122/130	exitPopup	Global	유지(전역 정당)
BaseCharacter:147/448	Healing_Aura VFX	Global	유지/파티

문서 정정 ①: 기존 "3b StandingImagesAtlas(436MB)"는 Phase 0.5c에서 아틀라스가 이미 해체돼 stale. 실제 대상은 개별 스프라이트 로드 2곳(UI_Info:95, UI_EscapeMenu:47). 정정 ②: "SceneDataSO.cs 삭제"는 취소(PreloadSO 5종 베이스, 실사용).

Before — 2버킷 (애매하면 Global 도피)



After — 다층 수명 (제 수명 스코프로)



[도식 1] 2버킷 도피 구조 → 실제 사용 패턴대로의 다층 수명 재배포.

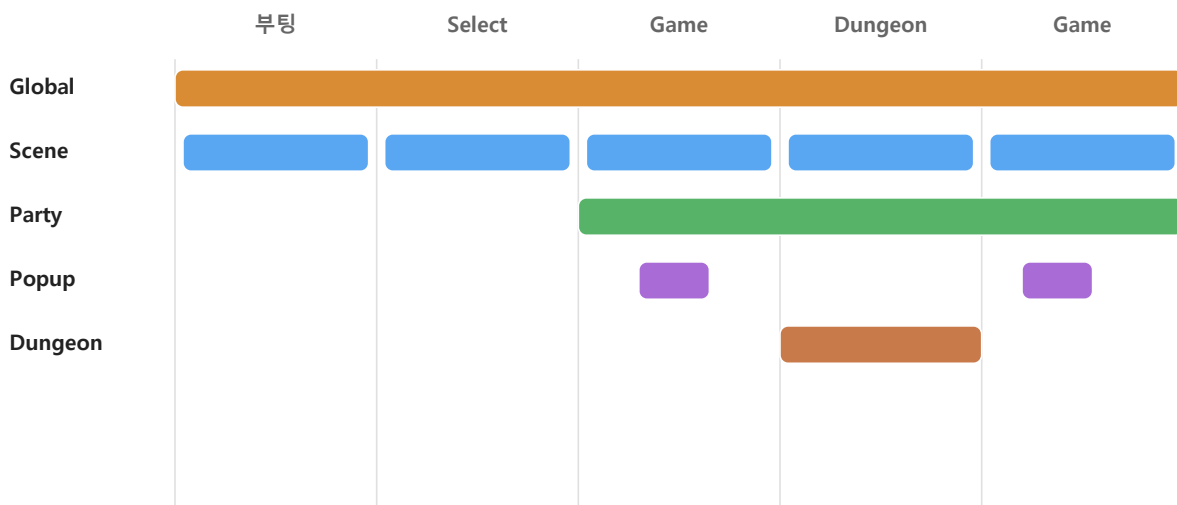
2. 새 수명 계층 설계

현재 ResourceScopeType = { Global, Scene } 2개뿐. 실제 사용 패턴대로 계층을 신설한다:

```
enum ResourceScopeType { Global, Scene, Party, Popup, Dungeon }
```

스코프	수명 (생성 → Dispose 혹)	담는 것	구분
Global	부팅 → 종료 (Dispose 안 함)	공용 VFX·exitPopup·SceneTable	기존
Scene	진입 → 전환 (ChangeSceneScope)	맵·씬 UI·팝업 프리팹(3a)	기존 자동
Party	파티 구성 → 해체/교체	인게임 캐릭 프리팹(3c)	신설·수동
Popup	팝업 open → close	스탠딩 등 대형 콘텐츠(3b)	신설·수동
Dungeon	입장 → 퇴장	던전 산물·루트 UI(3d)	신설·수동

- CreateScope(type, name) / GetScope(type) 이미 존재 → 인프라 재사용.
- 새 스코프의 난점 = "언제 Dispose하나"(혹 위치). 씬처럼 자동이 아니라 각 도메인 경계에 수동으로 건다.
- Global은 절대 Dispose 안 함(불변). Scene은 전환 시 자동 회전(불변).



↑ Global=불변 · Scene=전환마다 자동 회전 · Party/Popup/Dungeon=도메인 경계에서 생성·Dispose

[도식 2] 스코프 생명주기 — Global은 전 구간 상주, Scene은 씬마다 회전, Party/Popup/Dungeon은 도메인 경계에서 생성·Dispose.

3. 재배포 명세

3a. 팝업 프리팹 → 씬 스코프 ★ 첫 타자

대상: UIManager.ShowPopupUIAsync. 변경 = isGlobal:true 삭제(기본 false=Scene) + 메서드를 try/finally로 감싸 _isLoadingPopup 복구 보장.

Before

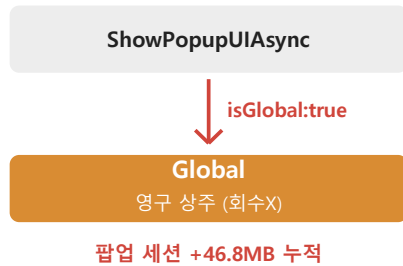
```
_isLoadingPopup = true;
...
prefab = await ...LoadAsync<GameObject>(key, isGlobal: true);
if (prefab == null) return null; // _isLoadingPopup 미복구=영구 차단
...
_isLoadingPopup = false;
```

After

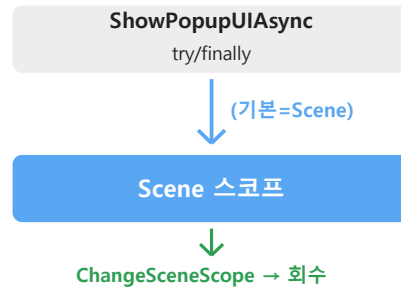
```
_isLoadingPopup = true;
try {
    prefab = await ...LoadAsync<GameObject>(key); // isGlobal 제거=Scene
    if (prefab == null) return null;
    ... return popup;
} finally { _isLoadingPopup = false; } // 실패·예외에도 복구
```

- 정책 A(씬 스코프): 씬당 첫 오픈 1회만 히치, 씬 전환 시 회수. 작은 팝업엔 최적.
- 목표: 팝업 세션 영구증가 +46.8MB → 0. blast radius = 이 메서드 하나.

Before



After



[도식 3] 팝업 로드 Before(Global 영구) / After(Scene → 전환 시 회수).

함정: 팝업 프리팹이 대형 텍스처를 직접 참조(UpgradePanel 245MB·UI_Info 217MB·ItemInfo 172MB)하면, 프리팹 핸들 회수 시 그 텍스처가 함께 반납되는지 텍스처 메모리로 확인.

3b. 스탠딩 이미지 → 팝업 스코프 (새 스코프 신설)

대상: UI_Info:95, UI_EscapeMenu:47. 스탠딩은 크고 한 캐릭만 열람 → 씬 전환까지 드는 정책 A보다 팝업 닫을 때 회수(정책 B)가 유리. 여기서 팝업 수명 스코프를 신설한다.

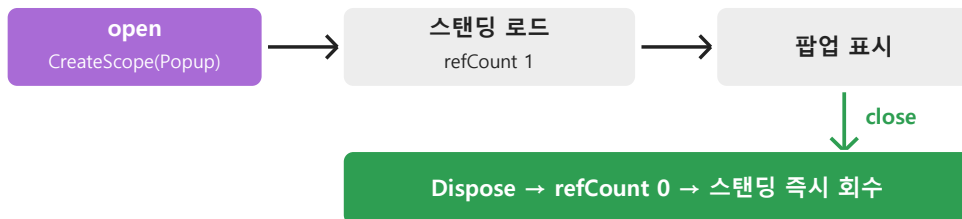
신설 후 (UIManager)

```
// 열 때: 팝업 전용 스코프 생성
var s = Managers.Resource.CreateScope(ResourceScopeType.Popup, $"Popup:{key}");
// 닫을 때(ClosePopupUI): Dispose → 스탠딩 즉시 회수
Managers.Resource.GetScope(ResourceScopeType.Popup)?.Dispose();
```

소비 측 (UI_Info)

```
// Before: isGlobal:true → Global 영구
// After: 팝업 스코프로 로드
var standing = await Managers.Resource.GetScope(ResourceScopeType.Popup)
    .LoadAsync<Sprite>(key, token);
```

- 설계 결정: 팝업 스택(중첩)이 있으므로 "Popup 스코프 1개 공유" vs "팝업마다 스코프" 결정 필요. 스택이 얕으면 최상위 1개로 단순화 권장(닫힐 때 그 스코프만 Dispose).
- 목표: 진입/세션 상위(UI_Info 217MB)를 열람 종료 시 회수.



대형 콘텐츠(스탠딩 217MB급)에 유리: 씬 전환까지 안 들고, 닫는 즉시 반납 (정책 B)

[도식 4] 팝업 스코프 open→close: Dispose가 refCount를 0으로 → 대형 스탠딩 즉시 반납.

3c. 파티 스코프 + Global 프리로드 다이어트

- 코드(파티 스코프): GameScene:172 인게임 캐릭 프리팹을 Global → Party 스코프. 파티원만 로드, 비파티 캐릭은 로드 자체 안 함. 파티 해체/교체 시 Dispose.
- Addressable 데이터(라벨 재분류): "Global" 라벨에서 *_Select(13종)·비파티 캐릭을 Select/Party 라벨로 이동. 코드 아님, 그룹/라벨 편집. 착수 전 "Global 라벨 내용물 감사" 필요.
- 목표: 진입 직후 Global 512.8MB 대폭 감소.

3d. 던전 스코프 + AtlasSpriteCache 스코프화 (우선순위 낮음)

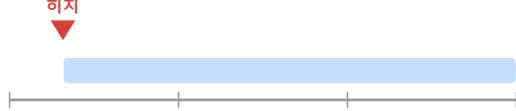
- GetSpriteFromAtlasAsync:177이 아틀라스를 isGlobal:true + _atlasSpriteCache 영구. 소비 아틀라스는 아이콘류 8개=이미지 0.5~4MB 최적이라 이득 작음.
- 던전 산물 → Dungeon 스코프, 캐시를 아틀라스 핸들 수명에 동행(R3/R5 합류). 목표: 왕복 +18.7MB → 0, AtlasSpriteCache 268 상주 해소.

4. 정책 결정 — 히치 vs 상주 (포트폴리오 재료)

lazy화하면 첫 오픈 히치가 생긴다. 어느 스코프에 둘지 = 히치와 상주의 트레이드오프이고, 이 판단을 측정으로 내리는 과정 자체가 어필 포인트다.

정책	수명	히치	상주	적합
A. 씬 스코프	씬당 유지	씬당 첫 1회	씬 동안	작은 팝업(3a)
B. 팝업 스코프	열림 동안만	열 때마다	최소	대형 콘텐츠(3b)

A. 씬 스코프 — 씬당 첫 1회 히치, 씬 동안 상주



B. 팝업 스코프 — 열 때마다 히치, 상주 최소



- A: 히치 적음 / 상주 큼 → 작은 팝업 프리팝(3a)
- B: 히치 잦음 / 상주 최소 → 대형 스탠딩·아틀라스(3b)
- 결정 근거 = [UIMetric] 실측 (히치 ms) + 메모리 리포트 (상주 MB)

[도식 5] 정책 A vs B — 대형(스탠딩/아틀라스)엔 B 선별 적용하는 하이브리드가 기본 방향. 결정 근거는 [UIMetric] 히치 실측 + 메모리 리포트 상주.

5. 실행 순서 + 검증

각 단계 = 독립 커밋 + 디버그창 체크포인트 버튼으로 Before/After 측정. 매 단계 Scene 버킷 불변(회귀 없음)을 R2/S1 계측이 자동 확인.

1. 3a 팝업 → 씬 스코프 (+_isLoadingPopup try/finally) — 새 스코프 0, 가장 쉬움. 팝업 세션 +46.8MB→0 첫 성과.
2. 3b 스탠딩 → 팝업 스코프 신설 — 첫 "새 스코프+Dispose 혹". 히치 vs 상주 측정 문서화.
3. 3c 파티 스코프 + 프리로드 라벨 다이어트 — 코드+Addressable 병행. 진입 512.8MB 감소.
4. 3d 던전 스코프 + AtlasSpriteCache(R3/R5 합류) — 이득 작아 마지막.

성공 기준 (Baseline v2 대비)

지표	Before v2	After 목표
팝업 세션 후 영구 증가	+46.8 MB	0
진입 직후 Global 핸들 메모리	512.8 MB	대폭 감소
던전 왕복 후 영구 증가	+18.7 MB	0
AtlasSpriteCache	268 상주	소비자 수명 동행
Scene 버킷	정상(96~101MB)	불변(회귀 없음)
UI_Info 첫 오픈 최악 프레임	123.4ms	악화 없음 + 원인 조사

완료 기준: 위 After 달성 + 개발 빌드 실측 1회.

6. 주의 / 함정

- Global 불변: Global 스코프는 절대 Dispose 안 함. 재배치는 "Global에서 빼는" 한 방향뿐.
- 새 스코프 Dispose 혹 누락 = 누수: Party/Popup/Dungeon은 자동 회전이 아님. 누락 시 LogAliveReport에 [?] 버킷(소유 스코프 없는 고아)으로 잡힘 = 검출됨.

- 팝업 중첩: Popup 스코프를 스택과 어떻게 맞출지 먼저 결정(권장: 최상위 1개).
- 프리팹→텍스처 의존: 프리팹 회수 시 직접 참조 대형 텍스처가 함께 반납되는지 텍스처 메모리로 확인.
- 에디터 열림 시 배치 컴파일 불가 / PS 커밋 메시지 따옴표는 -F 파일 경유.